Git Revert vs Git Rebase - Beginner Guide with Examples

This guide explains the difference between git revert and git rebase with practical examples.

Feature	git revert	git rebase
Purpose	Undo changes by creating a new commit	Rewrite commit history by replaying commits
History	Preserved	Rewritten
Safe on shared branch?	Yes	Usually No

1. git revert

Command:
`git revert <commit-id>`

Example history:
A -> B -> C -> D (HEAD)
If commit C introduced a bug:
`git revert C`
Result:
A -> B -> C -> D -> E
E is a new commit that reverses C. The original history remains intact.

2. git rebase

Command:
`git rebase main`

Before:
main: A -> B -> C
feature: D -> E (branched from B)

After rebasing feature onto main:
A -> B -> C -> D' -> E'
Git creates rewritten commits D' and E' to produce a linear history.

Conflict Resolution

If a conflict occurs during rebase:
```
git add .
git rebase --continue
```

To cancel the rebase:
`git rebase --abort`

When to Use

Use **git revert** on shared/public branches such as main.
Use **git rebase** to clean up your local commit history before creating a pull request.

Interview Questions

1. Difference between revert and reset?
2. Why is revert safer on main?
3. Difference between merge and rebase?
4. What does interactive rebase do?
5. How do you resolve rebase conflicts?